

Veriducta MVP: Detailed Build Proposal

What the MVP Is

The MVP is not a product launch. It is a technically rigorous proof of concept that demonstrates one claim no existing RAG observability tool can make: given a failed answer, Veriducta can identify which pipeline stage caused the failure and quantify how much that stage contributed to the quality loss. The MVP is complete when three things are true. First, the full pipeline runs end to end over a real document corpus and produces structured, claim-level answers with verification. Second, the causal replay engine correctly identifies the root cause stage on the synthetic corruption benchmark with measurable accuracy. Third, every metric in the reliability scorecard is computed, reproducible, and comparable against a RAGAS baseline.

Scope Boundaries

Included in MVP

A RAG pipeline over 30 to 50 public engineering and geoscience documents. Hybrid BM25 plus dense retrieval with cross-encoder reranking. Structured claim-level JSON output enforced by schema validation. NLI entailment checking with 3-class heuristic output. Single-pass BM25 counterevidence retrieval using entity-expanded contrastive queries. A pre-annotated version graph for temporal validity filtering. A four-stage gold ablation engine covering chunking, retrieval, reranking, and generation. A 60-case synthetic corruption benchmark with known ground-truth root cause labels. A 40-question golden QA dataset with full annotation. OpenTelemetry tracing across all pipeline stages. A Prometheus metrics endpoint and Grafana dashboard. A CI regression gate using relative regression detection. A full evaluation harness computing all scorecard metrics. A RAGAS baseline comparison. A public GitHub repository with complete documentation and one technical blog post.

Excluded from MVP

Automated PDF date parsing. Multi-tenant SaaS infrastructure. Enterprise SSO or authentication. A reusable SDK or framework adapters. LangChain or LlamaIndex integrations. Well log LAS/DLIS parsing. Image, figure, or stratigraphic column handling. Real-time document ingestion pipelines. A polished web UI beyond the Grafana dashboard. Oracle-free production debugging for novel queries beyond heuristic span signals.

Build Phases

The MVP builds in six phases across twelve weeks. Each phase has a clear entry condition, a set of deliverables, and an exit condition that must be satisfied before the next phase begins.

Phase 0 — Pre-Project Setup (Before Week 1)

Entry Condition

None. This phase happens before the clock starts.

Objectives

Validate that the corpus selection and annotation are feasible before committing to the twelve-week timeline. Discover scope problems early rather than in Week 6.

Tasks

Corpus screening. Review all candidate documents from USGS Publications Warehouse, Kansas Geological Survey, NIST, and OSHA. For each document, verify: explicit effective date readable

without regex, known supersession relationships, text-dominant content, and open access licensing. Flag any document that fails any criterion. Target 50 candidate documents, expecting 30 to 40 to pass screening.

Annotation pilot. Annotate 10 golden QA questions completely, including supporting chunk IDs, counterevidence chunk IDs, temporal validity tag, difficulty label, and failure mode label. Time each annotation. If average annotation time exceeds 45 minutes per question, reduce the golden QA target from 40 to 30. If it exceeds 60 minutes, reconsider the corpus — the domain knowledge required may be too deep for the available time.

Environment setup. Install Docker, set up local Qdrant, MinIO, Prometheus, Grafana, and the OpenTelemetry Collector using Docker Compose. Verify all services are reachable. Create the GitHub repository with the directory structure defined in the master documentation. Install all Python dependencies and verify imports. Confirm the Anthropic API key is working with a simple Claude Sonnet 4.6 call.

Toolchain smoke test. Run PyMuPDF on three representative corpus documents. Verify text extraction quality. Run pdfplumber on the same three documents and verify table extraction produces clean CSV. Confirm rank-bm25 indexes a small test corpus and returns ranked results. Confirm qdrant-client can upsert and retrieve a small set of test vectors.

Deliverables

- 30 to 50 screened and approved corpus documents with PDF files downloaded
- 10 completed golden QA annotations with timing data
- Revised annotation target based on timing data
- Fully running local development environment
- GitHub repository with initial directory structure and README skeleton
- Toolchain smoke test results documented

Exit Condition

All 10 pilot annotations complete. Development environment fully operational. At least 30 documents pass corpus screening. Toolchain smoke tests pass. Annotation time per question is understood and the 40-question target is confirmed or revised.

Phase 1 — Ingestion Pipeline (Weeks 1 to 2)

Entry Condition

Phase 0 complete. All corpus documents downloaded. Development environment operational.

Objectives

Build a complete, versioned, replayable ingestion pipeline. Every document in the corpus is parsed, chunked, embedded, and stored in Qdrant with full metadata. Every ingestion decision is captured in a configuration snapshot that can be exactly reproduced.

Week 1 Tasks

Metadata sidecar files. Create the JSON sidecar file for every corpus document following the schema in the master documentation. Fill in document ID, title, source, document type, effective date, expiry date, supersedes, superseded by, version hash, filename, page count, has tables, has figures, figures in scope, primary entities, and chunking variant. Validate every sidecar file against the schema using jsonschema. No document enters the ingestion pipeline without a validated sidecar.

PDF parsing. Implement veriducta/ingestion/parser.py. Use fitz.open() for text extraction per page, preserving page number and positional metadata. Use pdfplumber for table detection on the same pages. For each detected table, extract as CSV and linearise to Markdown following the format

defined in the master documentation. Build the `ParsedDocument` dataclass containing per-page text blocks, linearised tables, and page metadata.

Chunker implementation. Implement `veriducta/ingestion/chunker.py`. Hierarchical parent-child strategy: section detection using the configurable `section_boundary_markers` regex patterns, parent chunk assembly at 1400 to 1600 tokens, child chunk sliding window at 200 to 400 tokens with 50-token overlap. When `boundary_aware: true`, the chunker must never split a child chunk across a detected section boundary — if a window would cross a boundary marker, it terminates at the boundary and the next chunk starts after it. Assign chunk IDs as `{document_id}-ch-{zero_padded_index}` and parent IDs as `{document_id}-par-{zero_padded_index}`. Serialise the chunking configuration as a snapshot JSON file, compute its hash, and store the snapshot at `config/chunking_snapshots/{hash}.json`.

Version graph. Implement `veriducta/ingestion/version_graph.py`. Build a `networkx.DiGraph` from all sidecar files. Add each document as a node with `effective_date` attribute. Add directed edges from each `supersedes` entry pointing to the current document with `supersession_date` equal to the current document's effective date. Implement `get_valid_documents(query_date)`, `get_superseded_documents(query_date)`, and `get_supersession_chain(document_id)`. Serialise the graph to `corpus/version_graph.json` using `networkx.node_link_data()`.

Week 2 Tasks

Qdrant schema and upsert. Create the `veriducta_chunks` collection in Qdrant with cosine distance metric and the BAAI/bge-large-en-v1.5 embedding dimension (1024). Implement the embedding pipeline: load the model once, batch child chunks into groups of 32, embed each batch, upsert to Qdrant with the full payload schema defined in the master documentation. Verify upsert with a spot-check query on three known chunks.

Chunking failure corpus. Identify 10 to 15 documents from the screened corpus where a critical clause or exception appears within 50 tokens of a natural section boundary. For each, run ingestion twice: once with `boundary_aware: true` saving to `veriducta_chunks`, and once with `boundary_aware: false` saving to `veriducta_chunks_naive_{document_id}`. For each document, manually identify the specific child chunk IDs where the critical clause appears intact in the boundary-aware version and split across two chunks in the boundary-naive version. Record this mapping in `corpus/chunking_variants/{document_id}_boundary_map.json`.

BM25 index. Build the BM25 index over all child chunk texts using `rank_bm25.BM25Okapi`. Serialise to `corpus/bm25_index.pkl`. Verify with three test queries that the top-5 results are plausible.

Ingestion orchestrator. Implement `veriducta/ingestion/ingestor.py` as the CLI entry point that runs the full ingestion sequence: parse, chunk, embed, upsert, build version graph, build BM25 index. Implement `scripts/ingest_corpus.py` as the command-line wrapper. Verify the complete ingestion run finishes without errors and all documents are retrievable by document ID.

Unit tests. Write unit tests for: chunker boundary detection (verify that boundary-naive and boundary-aware produce different splits on a controlled input), sidecar schema validation (verify that malformed sidecars are rejected), version graph queries (verify `get_valid_documents` returns correct sets for test dates), and Qdrant upsert and retrieval (verify a known chunk is retrievable by vector similarity).

Deliverables

- All corpus documents parsed, chunked, and ingested into Qdrant
- Chunking failure corpus: 10 to 15 documents with both configuration snapshots stored
- BM25 index built and verified

- Version graph built and serialised
- All sidecar JSON files validated
- scripts/ingest_corpus.py runs to completion without errors
- Unit tests passing for ingestion components

Exit Condition

Run scripts/ingest_corpus.py from scratch on a clean Qdrant instance. All documents ingest without errors. A spot-check retrieval query on three known chunks returns the correct chunk in the top-3 results. The version graph correctly identifies documents valid and superseded for three manually chosen test dates. Unit tests pass.

Phase 2 — Retrieval Pipeline (Weeks 3 to 4)

Entry Condition

Phase 1 complete. All documents ingested. BM25 index and version graph operational.

Objectives

Build a fully instrumented hybrid retrieval pipeline that produces traceable, replayable ranked candidate sets. Every retrieval decision — BM25 scores, dense scores, RRF ranks, reranker scores, temporal filter decisions — is logged with sufficient detail for the causal replay engine to test counterfactuals without re-running expensive components.

Week 3 Tasks

BM25 retrieval module. Implement veriducta/retrieval/bm25_retriever.py. Load the serialised BM25 index at startup. Tokenise queries using the same tokeniser used at index time. Score all chunks and return the top-100 as a list of (chunk_id, bm25_score, bm25_rank) tuples. Apply temporal filtering: load the version graph, call get_valid_documents(query_date), filter the top-100 to only include chunks from valid documents, and log any rejected chunks as temporal filter rejections with their document ID and rejection reason.

Dense retrieval module. Implement veriducta/retrieval/dense_retriever.py. Load BAAI/bge-large-en-v1.5 at startup. Implement query embedding with the recommended prefix. Implement a simple in-memory LRU cache keyed on query text hash with TTL of 1 hour. Query Qdrant using collection.search() with limit=100 and a Qdrant payload filter for temporal validity on effective_date and expiry_date fields. Return top-100 scored candidates.

RRF fusion module. Implement veriducta/retrieval/fusion.py. Take BM25 top-100 and dense top-100 as inputs. Compute RRF scores using $1/(60 + \text{rank})$ for each list, assigning implicit rank 101 for candidates absent from a list. Merge and sort by RRF score. Return the unified ranked list of all unique candidates with their per-list scores and RRF score.

Temporal filter module. Implement veriducta/retrieval/temporal_filter.py as a standalone utility. Input is a list of chunk IDs and a query date. Output is a filtered list with a rejection log. A chunk is rejected if its effective_date is after the query date, or if its document has a superseding document whose effective_date is on or before the query date. Rejected chunks are logged with rejection_reason: "not_yet_effective" or rejection_reason: "superseded".

Week 4 Tasks

Cross-encoder reranker. Implement veriducta/retrieval/reranker.py. Load cross-encoder/ms-marco-MiniLM-L-12-v2 at startup. Take the top-40 RRF candidates as input. Run cross-encoder inference on all 40 query-chunk pairs in a single batch call. Return all 40 with their reranker scores and post-rerank ranks. This full pre-reranking candidate list with scores is stored in the retrieval trace — not just the

final selected set. This is what allows the replay engine to test reranker cutoff counterfactuals without re-running inference.

Parent-child expander. Implement `veriducta/retrieval/expander.py`. Take the top-8 post-rerank chunks as input. Fetch each chunk's `parent_chunk_id` from its Qdrant payload. Retrieve the parent chunk text from Qdrant. Construct the generation context as child chunk text followed by the parent section text with a section break marker. Return the expanded context with parent chunk IDs logged.

TraceableRetriever implementation. Implement `veriducta/retrieval/retriever.py` as the `VeriductaRetriever` class implementing the `TraceableRetriever` interface. The `retrieve()` method orchestrates the full sequence: BM25, dense, RRF, temporal filter, reranker, expander. It emits OpenTelemetry spans for each sub-stage. It writes a complete `RetrievalTrace` object to the evidence log. The `get_trace()` method reads a historical trace from the evidence log via the SQLite index. The `replay_with_config()` method re-runs retrieval using an override configuration snapshot, used by the causal replay engine.

Retrieval configuration snapshot. Serialise the retrieval configuration — BM25 top-k, dense top-k, RRF k constant, reranker model identifier, reranker top-k input, reranker top-k output, temporal filter enabled flag — as a versioned JSON snapshot with a computed hash.

Integration test. Run five queries over the ingested corpus end to end through the retrieval pipeline. Verify that: the temporal filter correctly excludes superseded document chunks, BM25 scores are non-zero for queries containing exact corpus terminology, dense retrieval returns semantically relevant chunks for paraphrase queries, the reranker re-orders at least one result relative to RRF for each test query, and the retrieval trace written to the evidence log contains all required fields.

Deliverables

- Full hybrid retrieval pipeline operational end to end
- Retrieval traces written to evidence log with all score fields populated
- Parent-child expansion working for all top-8 candidates
- Temporal filtering tested against version graph
- `VeriductaRetriever` implementing `TraceableRetriever` interface
- Integration tests passing for five representative queries

Exit Condition

Five representative queries run through the full retrieval pipeline without errors. Each trace in the evidence log contains BM25 scores, dense scores, RRF ranks, pre-reranking top-40 with scores, post-reranking top-8, and parent expansion log. Temporal filter correctly handles at least one superseded document case. Unit and integration tests pass.

Phase 3 — Generation and Verification (Weeks 5 to 6)

Entry Condition

Phase 2 complete. Retrieval pipeline operational and producing valid traces.

Objectives

Build a structured generation pipeline that enforces claim-level citation, verifies every claim by NLI entailment, and runs a single-pass counterevidence scan using the fully specified algorithm from the master documentation.

Week 5 Tasks

System prompt engineering. Write and iterate on the Claude Sonnet 4.6 system prompt. The prompt must: instruct the model to respond only with valid JSON matching the output schema, require `citation_chunk_id` for every claim, require `key_entities` for every claim with at least two specific

technical terms, prohibit prose outside the JSON object, and explain the confidence and temporal validity classification rules. Test the prompt on 10 representative retrieval contexts and verify schema compliance on the first try for at least 8 of 10. Iterate until 9 of 10 comply without retry.

Structured generator. Implement `veriducta/generation/generator.py`. Call the Anthropic API with the system prompt, the assembled context (child chunk texts plus parent section texts, labelled with chunk IDs), and `max_tokens: 2048`. Parse the JSON response. Validate against the output schema using `jsonschema`. If validation fails, retry up to two times with an additional correction instruction appended to the user message. If validation still fails after two retries, mark the query as a generation failure and log it. Compute and log input tokens, output tokens, estimated cost, and generation latency.

NLI entailment checker. Implement `veriducta/generation/entailment.py`. Load cross-encoder/`nli-deberta-v3-base`. For each claim, form a sentence pair with its `citation_chunk_id` chunk text. Run batch inference. Apply the 3-class heuristic: supported if entailment probability above 0.65, contradicted if contradiction probability above 0.85 and neutral below 0.30, ambiguous-conditional if neutral above 0.40 and contradiction between 0.30 and 0.70, unresolved otherwise. Update each claim's `verification_status`. Set `requires_expert_review: true` on any answer with at least one ambiguous-conditional or contradicted claim.

Generation trace. Implement generation trace logging following the schema in the master documentation. The trace must link to its parent retrieval trace via `retrieval_trace_id`. Log the full structured answer, the full verification report, all NLI probability distributions per claim, and all cost and latency fields.

Week 6 Tasks

Counterevidence retrieval — full implementation. Implement `veriducta/generation/counterevidence.py` following the five-step algorithm specified exactly in the master documentation.

Step 1: extract `key_entities` from all claims, flatten, deduplicate, remove domain-adapted stopwords.

Step 2: construct the contrastive query string by joining up to 10 entities followed by the fixed keyword expansion: exception OR limitation OR superseded OR contraindicated OR contradicts OR "shall not" OR "not applicable" OR warning OR caution.

Step 3: run BM25-only retrieval with temporal filtering, returning top-10 candidates. Log the contrastive query string and all 10 candidates with their BM25 scores.

Step 4: run NLI batch inference scoring all 10 candidates against all claims. Log all probability distributions.

Step 5: classify each candidate-claim pair and update verification statuses. Attach contradicting chunks and conditional conflict chunks to affected claims.

Handle the entity-sparse case: claims with fewer than two entries in `key_entities` receive `verification_status: "not_searched"` with the logged reason `"insufficient_entity_signal"`.

TraceableGenerator implementation. Implement `veriducta/generation/verifier.py` and `veriducta/generation/generator.py` as the `VeriductaGenerator` class implementing `TraceableGenerator`. The `generate()` method calls the Anthropic API and validates the schema. The `verify()` method runs entailment checking and counterevidence retrieval and returns the full `VerificationReport`. The `replay_with_context()` method accepts an override context or config and re-runs generation, used by Stage 2 and Stage 4 of the causal replay engine.

End-to-end pipeline test. Run the full pipeline — retrieval then generation and verification — on 10 queries from the pilot annotation set. Verify: all answers conform to the JSON schema, citation chunk

IDs reference chunks that actually exist in Qdrant, NLI entailment scores are populated for all claims, counterevidence scan runs and logs results for all queries with sufficient entity signal, generation traces are written to the evidence log and link to their retrieval traces.

Deliverables

- Structured generation with JSON schema enforcement and retry logic
- NLI entailment checking with 3-class heuristic output
- Single-pass counterevidence retrieval with full step-by-step logging
- VeriductaGenerator implementing TraceableGenerator interface
- Full pipeline running end to end on 10 pilot queries
- Generation traces written to evidence log linking to retrieval traces

Exit Condition

10 pilot queries run through the full pipeline. All answers conform to the JSON schema with no unhandled schema validation errors. At least 8 of 10 answers have citation_chunk_id fields referencing real Qdrant chunk IDs. Counterevidence scan completes for all 10 queries and logs at least one candidate per query. NLI scores are populated. Traces are linked. End-to-end latency for a typical query is below 10 seconds.

Phase 4 — Causal Replay Engine and Evaluation Dataset (Weeks 7 to 10)

This is the longest phase and the core technical contribution of the MVP. It runs four weeks because the evaluation dataset and the replay engine must be built in parallel and validated together.

Entry Condition

Phase 3 complete. Full pipeline operational end to end. Generation traces written to the evidence log.

Week 7 Tasks — Golden QA Dataset Completion

Complete the golden QA annotation. 10 questions were annotated in Phase 0. Complete the remaining 20 to 30 questions targeting the revised annotation count from the pilot timing data.

For each question, the complete annotation record must include: question text, correct answer text, all supporting chunk IDs (verified by manual lookup in Qdrant), all known counterevidence chunk IDs (verified by manual lookup), temporal validity tag, difficulty label, failure mode label from the defined set, domain tag, and annotator notes for any ambiguous cases.

Failure mode label assignment guide. Assign chunking_boundary when the supporting chunk IDs exist in the corpus but the critical content is at a section boundary. Assign retrieval_miss when retrieval would be expected to fail even with correct chunking. Assign omission when the counterevidence chunk IDs represent material exceptions the answer must acknowledge. Assign temporal_confusion when the question involves a document that has been superseded. Assign hallucination when the correct answer is not derivable from any chunk in the corpus. Assign contradiction when two corpus chunks make substantively incompatible claims on the question topic. Assign none when the question has a clean, unambiguous answer with no known failure modes.

Run all 40 annotated questions through the pipeline and log the results. This serves as the first baseline evaluation run.

Week 8 Tasks — OpenTelemetry Instrumentation and Evidence Log

Full OpenTelemetry instrumentation. Wrap every pipeline stage in named spans following the span hierarchy in the master documentation. Every span must carry: configuration snapshot hash, input hash, output hash, latency in milliseconds, and token counts where applicable. Use context propagation to ensure child spans are correctly nested under their parent spans. Verify span hierarchy with a test run and inspection in the local OTLP collector.

Evidence log with rotation and SQLite index. Implement `veriducta/observability/evidence_log.py`. Write one JSON Lines entry per query to `evidence_logs/YYYY-MM-DD.jsonl`. Implement the gzip compression job for files older than 24 hours. Implement the SQLite index writer that records (`trace_id`, `log_file`, `byte_offset`, `query_hash`, `created_at`, `quality_score`, `flagged_as_failure`) for every entry. Implement the reader that retrieves a full trace by ID using the SQLite index for $O(1)$ lookup without full log scan.

Interface documentation. Write the `TraceableRetriever` and `TraceableGenerator` interface docstrings completely. Add a section to the README titled "Extending Veriducta to Your Own Pipeline" that explains what a LangChain adapter would need to implement. This does not require building the adapter — it requires showing that the interface boundary is designed for portability.

Prometheus metrics. Implement all metric definitions from the master documentation in `veriducta/observability/metrics.py`. Verify the Prometheus endpoint at port 8000 returns all metric families. Import the Grafana dashboard JSON and verify panels display data from the first evaluation run.

Week 9 Tasks — Synthetic Corruption Benchmark

Build all 60 synthetic corruption cases following the distribution in the master documentation.

Retrieval corruptions — 20 cases. For swap corruptions, select a base question from the golden QA set, identify its correct top-1 chunk, and find a semantically plausible but incorrect chunk from the same document type. Manually construct a modified retrieval trace where the incorrect chunk is substituted for the correct one. For supersession removal corruptions, temporarily remove the superseding document from Qdrant and the version graph, re-run retrieval, and capture the degraded trace. For BM25 zeroing, set the BM25 weight to zero in the RRF configuration snapshot and re-run. For top-k reduction, set `reranker_top_k_output` to 1 and re-run.

Chunking corruptions — 15 cases. For each selected document in the chunking failure corpus subset, activate the boundary-naive collection by modifying the retrieval configuration to query `veriducta_chunks_naive_{document_id}` instead of `veriducta_chunks`. Re-run the full pipeline. Verify that retrieval fails to return the split clause chunk in the top-8. At least 10 of the 15 must use documents where the split occurs mid-sentence or at an ambiguous entity boundary rather than at a clean paragraph break.

Reranker corruptions — 15 cases. For top-1 forcing corruptions, use the stored pre-reranking candidate list from a historical trace and truncate to rank-1 before constructing the generation context. For cross-encoder bypass, replace the reranker scores with RRF scores in the historical trace and re-run generation. For score inversion, swap the scores of ranks 1 and 3 in the pre-reranking list and re-run generation.

Generation corruptions — 10 cases. For unstructured prompt corruptions, replace the structured system prompt hash in the generation configuration snapshot with a prompt that asks for a prose answer without citation enforcement. For contradictory instruction injection, append a contradictory instruction to the system prompt. For token limit truncation, set `max_tokens` to 200 in the configuration snapshot to force JSON truncation.

Each of the 60 cases is serialised to `data/synthetic_corruptions/corruptions.jsonl` with the schema defined in the master documentation including the `ground_truth_root_cause` field and the `is_realistic_boundary_error` field.

Week 10 Tasks — Causal Replay Engine Implementation

Four-stage gold ablation engine. Implement `veriducta/replay/ablation.py`. The engine takes a `trace_id` and the corresponding `question_id` from the golden QA set as inputs.

Stage 1 — chunking ablation: check if the question's primary document is in the chunking failure corpus. If yes, call `retriever.replay_with_config()` with the boundary-aware chunking collection. Compute Recall@5 delta against the historical retrieval. Log the result including which specific chunk was recovered or not recovered.

Stage 2 — retrieval ablation: load the ground-truth supporting_chunk_ids from the golden QA annotation. Construct a mock retrieval context using the gold chunks. Call `generator.replay_with_context()` with the gold context and the historical generation configuration. Score the result using the answer quality metric. Compute the quality delta against the historical answer.

Stage 3 — reranker ablation: load the `pre_rerank_top40` from the historical retrieval trace. Re-construct generation contexts using top-1, top-3, top-5, and top-8 cutoffs (without re-running the reranker). Call `generator.replay_with_context()` four times. Compute quality deltas for each cutoff.

Stage 4 — generation ablation: load the baseline system prompt. Call `generator.replay_with_context()` with the historical retrieval context and the baseline prompt. Compute quality delta.

Produce the `ReplayReport` dataclass with per-stage attribution results and the primary root cause label.

Heuristic span attribution. Implement `veriducta/replay/heuristic.py`. Read the generation trace and apply the three heuristic signals: maximum reranker score threshold, temporal validity flag on retrieved chunks, and overall NLI entailment rate. Return a `HeuristicSignalReport` with all three signals and the explicit disclaimer language.

Corruption runner. Implement `veriducta/replay/corruption.py`. Iterates over all 60 synthetic corruption cases. For each case, runs the gold ablation engine and records the attributed root cause stage. Computes root-cause localization accuracy overall and separately for the `is_realistic_boundary_error: true` subset.

Deliverables

- 40-question golden QA dataset complete and validated
- 60-case synthetic corruption benchmark built
- OpenTelemetry instrumentation across all stages
- Evidence log with rotation and SQLite index
- Prometheus metrics endpoint operational
- Grafana dashboard displaying live metrics
- Four-stage gold ablation engine implemented
- Heuristic span attribution implemented
- Synthetic corruption runner implemented
- Root-cause localization accuracy computed on first benchmark run

Exit Condition

The synthetic corruption runner completes all 60 cases without errors. Root-cause localization accuracy is computed and logged. At least one chunking corruption case is correctly attributed to the chunking stage by Stage 1 ablation. At least one retrieval corruption case is correctly attributed to the retrieval stage by Stage 2 ablation. The Prometheus endpoint returns all defined metrics. The Grafana dashboard renders data from the evaluation run. The SQLite index supports O(1) trace retrieval by ID.

Phase 5 — Evaluation Harness and Regression Gate (Weeks 11 to 12)

Entry Condition

Phase 4 complete. Golden QA dataset complete. Synthetic corruption benchmark built and running. Replay engine producing attribution reports.

Week 11 Tasks

Evaluation runner. Implement `veriducta/evaluation/runner.py`. The runner takes the golden QA dataset and the pipeline configuration as inputs. For each of the 40 gold questions, it runs the full pipeline and collects the structured answer, verification report, and generation trace. For each of the 60 synthetic corruption cases, it runs the gold ablation engine and records the attributed root cause. It passes all results to the metrics computation module.

Metrics computation. Implement `veriducta/evaluation/metrics.py`. Compute all metrics defined in the master documentation:

Retrieval metrics: Recall@5, Recall@10, MRR, nDCG@10, temporal-valid retrieval rate, evidence diversity. These require matching retrieved chunk IDs against `supporting_chunk_ids` in the golden QA annotations.

Answer quality metrics: claim accuracy using NLI entailment of each claim against the ground-truth answer text, citation entailment rate from the generation verification report, omission rate computed as the fraction of questions where known counterevidence `chunk_ids` appear in the retrieved set but the answer does not acknowledge them, contradiction acknowledgment rate computed as the fraction of questions with failure mode label contradiction where the system flags at least one contradicted claim.

Causal attribution metrics: root-cause localization accuracy overall and on the realistic boundary-error subset, chunking ablation recovery rate, mean quality delta per ablation stage.

Calibration metrics: expected calibration error by mapping confidence tags to probability estimates and comparing against empirical claim accuracy, abstention precision.

Operations metrics: p50, p95, p99 latency from OpenTelemetry span data, cost per query from generation traces, cache hit rate.

RAGAS baseline. Run RAGAS over the same 40 gold questions using the same pipeline outputs. Compute RAGAS faithfulness, answer relevancy, context precision, and context recall. Include RAGAS baseline numbers in the evaluation report alongside Veriducta's metrics. The comparison shows what Veriducta measures that RAGAS cannot: omission rate, causal attribution accuracy, temporal validity rate, and contradiction acknowledgment rate.

Evaluation report writer. Implement `veriducta/evaluation/report.py`. Writes `evaluation_report_{timestamp}.json` with all metric values, the RAGAS comparison, and the per-question result log. Writes `evaluation_summary_{timestamp}.txt` with the CI-readable pass/fail table shown in the master documentation.

CI baseline. Run the evaluation suite once on the fully built pipeline. Write the metric values as `ci_baseline.json`. This is the reference point for all future regression gate comparisons.

Week 12 Tasks

CI regression gate. Implement `scripts/check_regression_gate.py`. Reads the current evaluation report and `ci_baseline.json`. Computes deltas for the five blocking conditions: faithfulness drop, recall@5 drop, p95 latency increase, root-cause localization accuracy drop, and unauthorized evidence exposure rate. Prints the structured diff table. Exits with code 1 if any blocking condition is triggered, code 0 otherwise.

Implement `.github/workflows/regression_gate.yml` following the workflow definition in the master documentation. Verify the gate passes on a clean run and fails when a deliberate regression is injected.

README and documentation. Write the complete project README including: project overview and tagline, the three problems being solved, architecture diagram, installation instructions, corpus description, evaluation scorecard with actual numbers from the final evaluation run, RAGAS comparison table, the known limitations section verbatim from the master documentation, and instructions for running the evaluation suite.

Technical blog post. Write one technical blog post titled "How Veriducta Found a Chunking Bug That Fooled Every Faithfulness Score." The post walks through a specific worked example from the synthetic corruption benchmark: a document where the boundary-naive chunking configuration splits a critical safety exception, the pipeline produces an answer that scores 0.84 on RAGAS faithfulness but is materially incomplete, and Stage 1 of the gold ablation engine correctly identifies the chunking boundary as the root cause. The post shows the full attribution report output and explains why this failure is invisible to standard faithfulness metrics.

Portfolio presentation. Prepare a 10 to 15 minute technical walkthrough covering: the problem and why existing tools miss it, the architecture with the five-layer diagram, a live demo running one query end to end through the pipeline, the evaluation scorecard with all metrics, the worked synthetic corruption example from the blog post, and the known limitations with the roadmap for closing them.

Deliverables

- Full evaluation harness computing all metrics on 40 gold questions and 60 synthetic cases
- RAGAS baseline comparison computed
- Evaluation report JSON and summary text
- CI baseline JSON file
- GitHub Actions regression gate workflow passing
- Complete project README with actual evaluation numbers
- Technical blog post published or ready to publish
- Portfolio presentation prepared

Exit Condition

The evaluation runner completes all 100 cases without errors. All metrics are computed and included in the evaluation report. The RAGAS comparison is present. The CI regression gate passes on the current codebase. The README renders correctly on GitHub. The blog post is written.

Key Risks and Mitigations

Risk: Annotation pace slower than expected. Mitigation: Phase 0 pilot annotation calibrates pace before the timeline starts. If pace exceeds 45 minutes per question, the target drops to 30 questions before Phase 1 begins.

Risk: NLI model performs poorly on technical prose. Mitigation: The 3-class heuristic with conservative thresholds and the ambiguous-conditional safety valve are specifically designed for this. The evaluation measures contradiction acknowledgment rate and abstention precision, which will surface systematic NLI failures. If NLI precision on the gold set is below 0.60, the entailment thresholds are tuned before the final evaluation run.

Risk: Causal replay engine produces incorrect attributions on chunking cases. Mitigation: The chunking failure corpus is built with manually verified boundary maps. If Stage 1 attribution accuracy on chunking corruptions is below 0.60, the boundary detection logic in the chunker is debugged before the final benchmark run. The synthetic corruption benchmark is specifically designed to expose this failure.

Risk: Claude Sonnet 4.6 JSON schema compliance is inconsistent. Mitigation: The system prompt is tested on 10 representative contexts in Week 5 and iterated until 9 of 10 comply without retry. The retry logic handles the remaining cases. If compliance is still below 90% after two prompt iterations, the JSON mode API parameter is used as a fallback to enforce valid JSON at the API level before schema validation.

Risk: p95 latency exceeds 10 seconds. Mitigation: The biggest latency contributors are dense embedding inference and the cross-encoder reranker. Both run on CPU. If latency is unacceptable, query embedding caching, BM25-only retrieval for the counterevidence scan, and reducing reranker input from 40 to 20 candidates are the first three optimisations to apply in that order.

Success Criteria

The MVP is successful when all of the following are true.

The full pipeline runs end to end on all 40 gold questions without unhandled errors.

Root-cause localization accuracy on the synthetic corruption benchmark is above 0.70 overall and above 0.65 on the realistic boundary-error subset.

At least one chunking corruption case is correctly attributed to chunking by Stage 1 ablation, with the attribution report showing the specific split chunk and the quality delta from boundary-aware recovery.

The evaluation scorecard shows at least four metrics that RAGAS does not compute: omission rate, causal attribution accuracy, temporal-valid retrieval rate, and contradiction acknowledgment rate.

The CI regression gate passes on the final codebase and fails when a deliberate regression is injected in a test run.

The technical blog post contains a worked example with a real attribution report output showing a chunking failure that scored above 0.80 on RAGAS faithfulness but was correctly identified as materially incomplete by Veriducta.

The README accurately describes what the system does and does not do, with the known limitations section present and honest.